

向量检索索引算法 —— 一份"找人"的生动笔记

 **贯穿全文的比喻:** 想象你手里有一张照片,要在一座 千万人口的城市 里找到 长得最像照片里这个人的居民。每个"居民"就是一条**高维向量**,你手里的照片就是**查询向量(Query)**,"长得像"就是**向量相似度**。下面的每一种索引算法,都是这座城市里一种不同的"找人策略"。

0. 开篇铺垫:NNS 与 ANNS

0.1 最近邻搜索 NNS (Nearest Neighbor Search)

NNS 要做的事:给定一个查询向量,从数据集中**精确地**找出与它最相似的若干个向量。

 **比喻:**警察办案,要求**百分之百确认**找到的就是照片上的人——哪怕把全城人挨个看一遍。

广泛应用于:

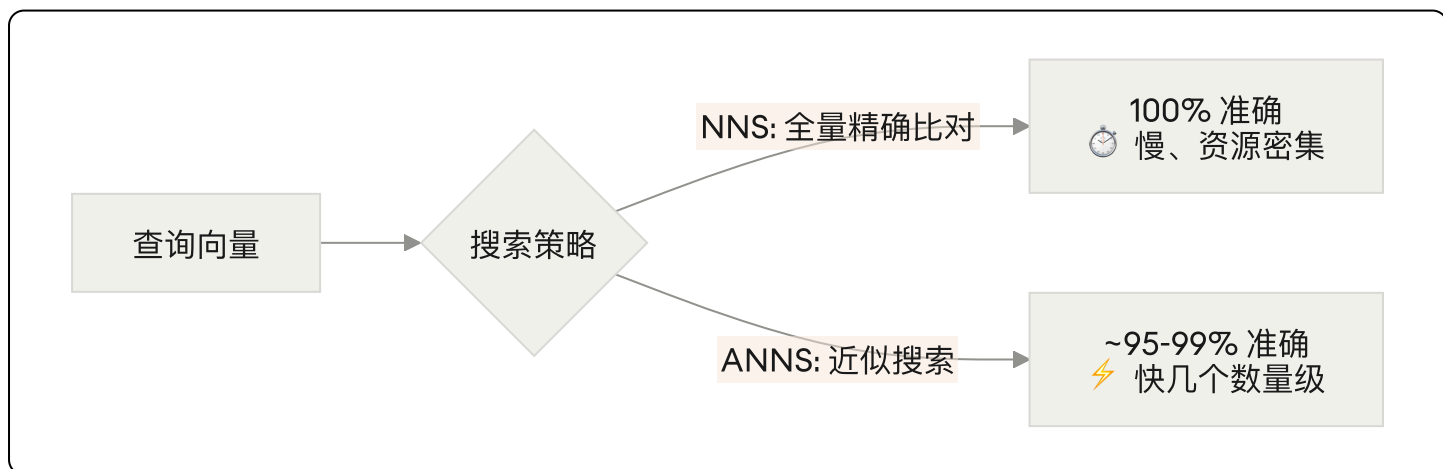
- 图像检索(以图搜图)
- 推荐系统(找相似用户/相似商品)
- 语音识别

致命问题:数据规模一大,逐个比对的耗时和资源消耗会**爆炸式增长**。给一千万个 768 维向量逐个算距离,每次查询都是一场灾难。

0.2 近似最近邻搜索 ANNS (Approximate NNS)

 **核心思想:**在可接受的精度范围内,牺牲一点点准确性,换取**数量级提升**的检索效率。它只需要找到目标向量的"近似邻居",而不必保证是绝对精确的最近邻。

 **比喻:**你不再要求"全城最像的那一个人",而是接受"找到一个**像到 99%**的人就行"。代价是偶尔会漏掉真正的第一名,但找人的速度从"三天"变成了"三秒"。



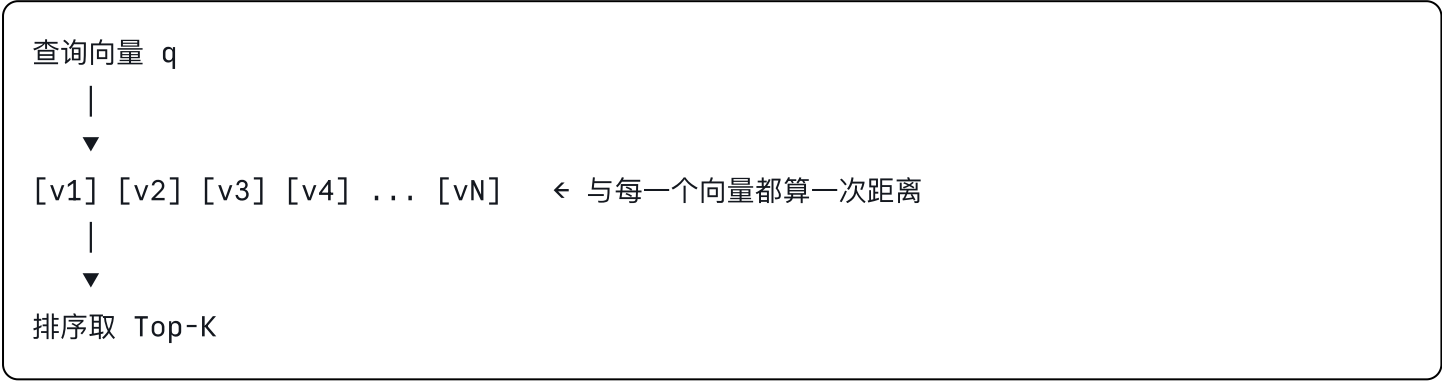
一句话总结:后面所有的索引算法(除 FLAT 外),本质都是 ANNS 的不同实现思路——有的靠"分区"缩小范围,有的靠"压缩"减少计算,有的靠"导航图"快速逼近。

1. FLAT —— 挨家挨户的暴力排查

一句话本质

- **比喻版:**拿着照片敲遍全城每一户的門,逐个对比。
- **技术版:**不建任何索引结构,对数据集做**暴力线性扫描(Brute-force)**,逐个计算查询向量与所有向量的距离。

核心原理



没有聚类、没有压缩、没有图——什么都没有,所以:

特性	表现
精度	✅ 100% 精确(召回率 1.0)
速度	❌ O(N),随数据量线性增长
内存	存储全部原始向量
构建成本	✅ 零构建成本(没有索引可建)

适用场景

- 🏠 小型数据集(百万级以内)且要求完全精确的搜索结果。 它也常被当作**基准(Ground Truth)**:其他 ANNS 算法的召回率,就是拿 FLAT 的结果来对照计算的。

🔧 例子:你的本地知识库只有 5 万条文档向量——FLAT 完全够用,既精确又省去了调参的麻烦。

2. IVF_FLAT 🌟 —— 先分区,再排查

一句话本质

- **比喻版:**先把全城划成若干个行政区,找人时只去照片上的人**最可能住的几个区**挨户排查。
- **技术版:**基于**倒排文件(Inverted File)**,先用聚类把向量空间划分成多个簇,查询时只在最近的几个簇内做线性搜索。

核心原理(三步走)

① 聚类(建索引时)

- 用聚类算法(典型如 **K-means**)把高维空间中的所有向量划分为多个子空间(簇)。
- 每个簇有一个代表向量——通常是簇的**中心点(centroid)**。

📌 比喻:按"居住地"给全城人分区,每个区有一个"区中心地标"。

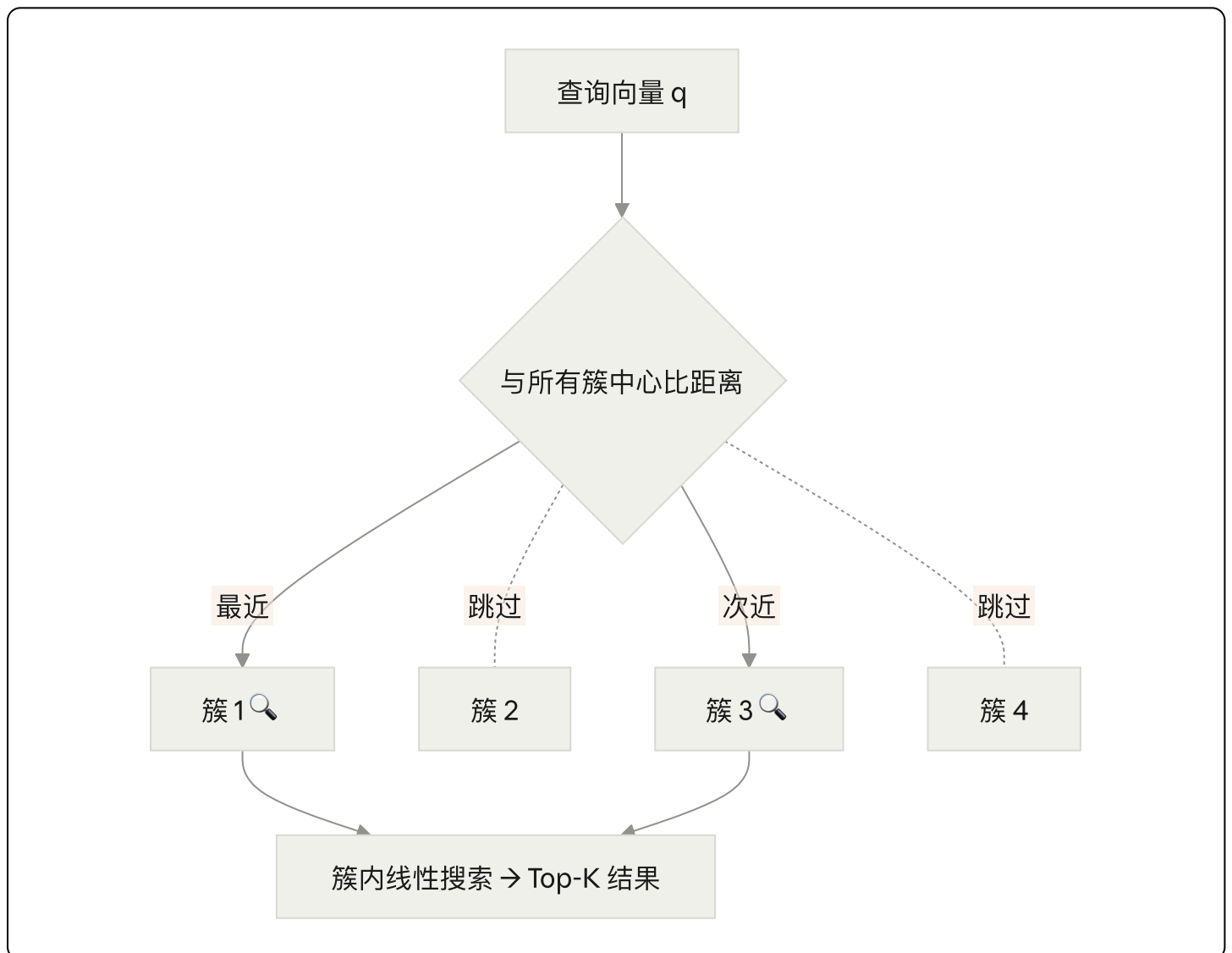
② 倒排索引

- 为每个簇建立倒排表: **簇中心 → [属于该簇的所有向量]**。
- 每个向量被映射到它所属的簇。

📌 比喻:每个区政府保存一份"本区居民花名册"。

③ 查询处理

1. 先把查询向量与**所有簇中心**比对,找出距离最近的簇。
2. 只在选中的簇内部做**线性搜索**(簇内仍是 FLAT 式的精确比对,所以叫 **IVF_FLAT**)。



关键参数:`nprobe`

`nprobe` 控制查询时要搜索多少个簇:

调整方向	效果
增大 <code>nprobe</code>	搜索更多簇 → 精确率 ↑, 查询时间 ↑
减小 <code>nprobe</code>	缩小搜索范围 → 计算时间 ↓, 牺牲部分精度

🔗 比喻:`nprobe=1` 只去"最可能的那个区"找;`nprobe=8` 把周边 8 个区都搜一遍——更稳妥,但更费时。

⚠️ 边界问题(为什么 `nprobe=1` 会丢人):如果目标向量恰好住在"两个区的交界处",它属于 A 区,但离 B 区中心更近的查询可能只搜了 B 区——这就是 IVF 召回率损失的主要来源,增大 `nprobe` 就是为了覆盖这种情况。

权衡与定位

- 没有量化操作 → 精度和存储开销都比较"保守"(向量原样存储),但搜索速度比 FLAT 快得多(只搜部分簇)。
- 通过划分子空间限制搜索范围,显著减少高维数据线性搜索的计算开销。

👉 适用于:需要在精度和查询速度之间取得平衡的场景,尤其是大规模、高维数据集。追求高精度、但能容忍一定查询延迟的应用!

3.IVF_SQ8 —— 分区 + 把档案压成"摘要卡"

一句话本质

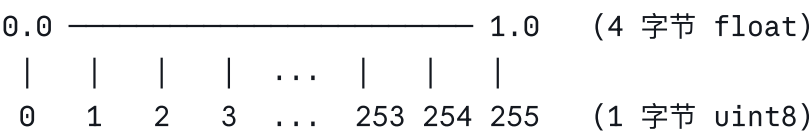
- 比喻版:还是分区找人,但每个人的详细档案被压缩成一张**简历摘要卡**——信息略有损失,但翻阅速度快、柜子省空间。
- 技术版:`IVF_SQ8 = IVF 倒排聚类 + SQ8 标量量化`。在 IVF_FLAT 基础上,用**标量量化(Scalar Quantization)**把每个维度的 4 字节浮点数压缩为 1 字节整数。

核心原理

① 标量化(Scalar Quantization)

把每个维度的连续浮点值,映射到一个小的整数范围(如 `uint8` 的 0~255):

原始值范围 [0.0, 1.0],按 256 个区间量化:



例: 0.4337 → 落入第 111 个区间 → 存为整数 111

📌 比喻:体重原本精确记录为 (65.4337 kg),量化后记成"65~66 kg 档"——丢了小数点后的细节,但找"体重 65 公斤左右的人"完全不受影响。

② 量化后的向量(Quantized Vectors)

- 每个维度用整数(如 uint8)表示,整数按原始值的分布划分到若干区间中。
- 存储空间直接降为原来的 1/4(4 字节 → 1 字节),查询时的计算量也随之下降。

③ 倒排索引与聚类

- 与 IVF_FLAT 相同:用聚类算法(如 K-means)把向量划分为多个簇,只是簇内存储和检索的都是量化后的向量。

收益与代价

维度	IVF_FLAT	IVF_SQ8
每维存储	4 字节 float	1 字节 int(↓75%)
精度	较高	略有损失(量化误差)
查询速度	快	更快(整数运算 + 更小内存带宽)

🌸 核心目标:在牺牲少量精度的前提下,大幅减少存储空间,并提升向量检索速度。

📌 例子:1000 万条 768 维向量,FLAT/IVF_FLAT 需要约 30 GB 存原始向量;换成 SQ8 后只需约 7.5 GB ——一台普通服务器的内存就装得下了。

4. IVF_PQ —— 分区 + 把档案"拆段编码"

一句话本质

- 比喻版:把每个人的档案拆成"外貌 / 职业 / 爱好"几个小段,每段不存原文,只存一个编号(指向该段的"标准模板");比时也只比对模板编号。
- 技术版:(IVF_PQ = 倒排文件 IVF + 乘积量化 PQ (Product Quantization)),用于加速大规模高维数据集的检索。

核心原理

① 倒排文件(IVF 部分)

- 理解为:先给向量分组,再去相关组里查找。
- 每个向量被分配到一个或多个倒排表中;查询时不遍历全部向量,而是先找到最相关的倒排表,再在其中搜索候选向量。
- 作用:减少搜索范围,提升检索速度。(与前两者相同,不再赘述)

② 乘积量化(PQ 部分)—— 本算法的灵魂

把一个高维向量拆成多个子段,每段单独压缩:

原始 768 维向量:

[—— 子段1 (96维) —— | —— 子段2 (96维) —— | ... | —— 子段8 (96维) ——]

在子空间1聚类出 256 个
"标准模板"(codebook)

▼
编号 17

在子空间2聚类出 256 个
"标准模板"(codebook)

▼
编号 203

在子空间8聚类出
256 个"标准模板"

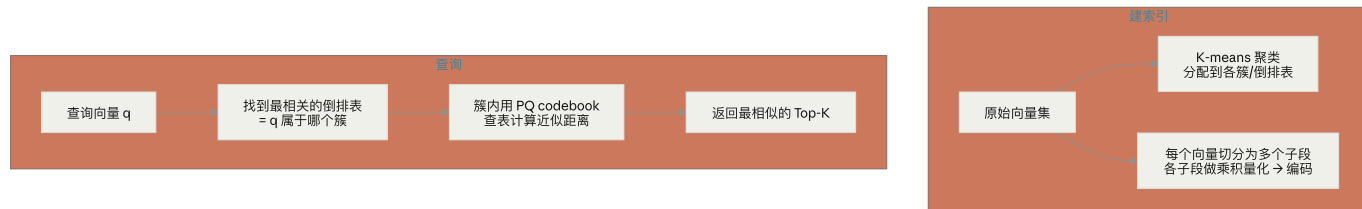
▼
编号 88

最终存储: [17, 203, ..., 88] ← 8 个字节,替代 768×4=3072 字节!

- 每个子空间生成对应的量化表示,即 **codebook(代码本)**。
- 原始向量不需要完整存储,只存压缩后的编码。
- 计算距离时,也不和原始向量直接计算,而是和量化后的中心点计算(可预先打表,查表即得距离)。
- 作用:极大减少存储空间、降低计算量、加快检索速度。

📌 **比喻细化:**描述一个人不再写完整档案,而是写"发型 17 号、职业 203 号、爱好 88 号"。比较两个人像不像,只需查"17 号发型 vs 42 号发型有多像"的预先算好的对照表——查表比现场观察快太多了。

③ IVF_PQ 完整流程



🌸 分工总结: **IVF**:负责把向量分组,减少搜索范围。 **PQ**:负责压缩向量,减少存储和计算成本。

与 IVF_SQ8 的对比理解

	SQ8(标量量化)	PQ(乘积量化)
压缩思路	每个维度单独压缩(float→int)	按子段整体压缩(段→codebook 编号)
压缩率	固定 4:1	可达几十比一(更激进)
精度损失	较小	较大
比喻	把数字"四舍五入"	把整段内容"换成标准模板编号"

🍰 适用于:超大规模数据集、内存极其受限、能容忍较明显精度损失的场景(如十亿级离线召回)。

5. HNSW 🌟 —— 高速公路 → 省道 → 小巷的逐层导航

一句话本质

- 比喻版:找人不再"分区排查",而是开导航——先上高速公路快速跨越大半个城市,再下到省道接近目标街区,最后在小巷里步行精确定位。
- 技术版:HNSW(Hierarchical Navigable Small World Graph,分层可导航小世界图)是一种基于图的索引算法,结合分层结构 + 小世界图理论,构建多层次图结构,逐层精细化搜索。

两大理论基石

① 跳表(Skip List)—— "分层"思想的来源

- 多层链表:底层是一个完整且有序的链表(存储所有元素);上层链表是下层的"抽象版",只包含部分元素,逐层变得更稀疏。
- 逐层查找:查询从最上层开始,当前层找不到就跳到下一层继续,直到最底层。

L3: 1 —————→ 50 ← 最稀疏,跨大步

L2: 1 —————→ 20 —————→ 50 → 77

L1: 1 → 9 —————→ 20 → 33 —————→ 50 → 77 → 92

L0: 1 → 5 → 9 → 14 → 20 → 27 → 33 → 41 → 50 → 62 → 77 → 85 → 92 ← 完整链表

查找 41: L3 走到 1→(50 太大,下沉) L2 走到 20→(50 太大,下沉)

 L1 走到 33→(50 太大,下沉) L0 从 33 走到 41 ✅

🚶 比喻:坐车先乘特快列车(只停大站),再换普快,最后步行到具体门牌号。

② 可导航小世界(NSW)—— "图"结构的来源

- 邻接列表:每个节点连接若干相似节点(邻接节点),每个节点都保存一份邻接列表。

- **遍历过程**:从随机选定的入口节点出发,沿着图的边**贪心地**走向更接近查询向量的邻居,逐渐逼近目标。

✂ 比喻:经典的**"六度分隔"**——你想认识某个陌生人,只需不断问"你的朋友里谁和这个人最像/最可能认识他",几跳之内就能找到。

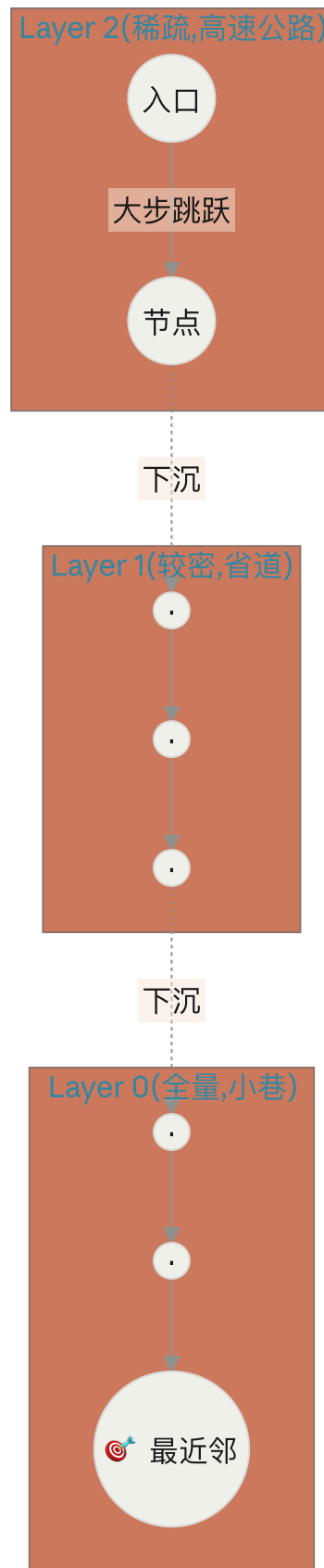
HNSW 的工作原理

索引构建

- **图的层次结构**:构建一个**多层图**——上层节点少,负责**粗粒度搜索**;底层节点多(包含全部向量),负责**精细搜索**。
- **连接邻居**:每个新加入的节点会被随机指定一个最高层级,并在它所在的每一层与若干个**近邻节点建立连接**——形成局部邻接关系,保证图结构稀疏的同时提升搜索效率。

查询过程

- **逐层搜索**:从最上层入口开始,每一层内通过贪心策略跳向更相似的邻居;本层无法更进一步时,**下沉**到下一层继续。
- **局部优化**:到达最底层后,在当前节点附近做**局部搜索**,遍历邻居节点,锁定最接近查询向量的结果。



HNSW = 多层图搜索 上层:快速缩小搜索范围 底层:精细查找最近邻 核心思想:像"导航"一样,从粗到细逐步接近目标向量。

权衡与定位

维度	表现
查询速度	✅ 极快(对数级跳跃逼近)
召回率	✅ 很高
内存占用	❌ 大——除了原始向量,还要存每个节点多层的邻接关系
构建成本	较高(逐点插入建图)

👉 适用于:对查询延迟和召回率都要求很高、且内存充足的在线服务场景(目前最主流的内存型 ANNS 索引之一)。

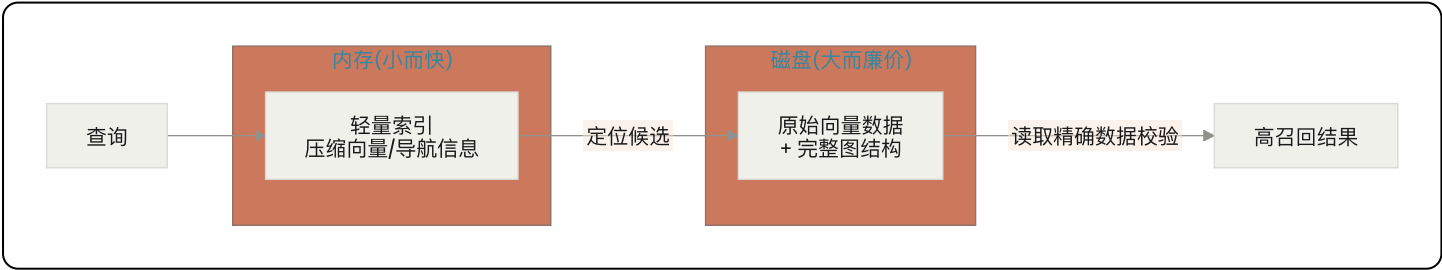
6. DiskANN —— 把地图放手边,把档案放仓库

一句话本质

- 比喻版:侦探手边只放一张轻薄的城市索引地图(内存),千万份详细居民档案放在仓库(磁盘);按图索骥,需要谁的档案才去仓库取那一份。
- 技术版:DiskANN 是一种基于磁盘的高性能向量近邻搜索算法,旨在解决大规模向量检索中的内存消耗问题。

核心思路

- 内存里:只放轻量级的索引结构(如压缩向量,用于快速导航)。
- 磁盘上:存放海量的原始向量数据 + 构建好的图结构。
- 查询时先用内存中的轻量索引规划路线,再按需从磁盘读取候选节点的完整数据做精确校验。



由此,DiskANN 能在保持高召回率和低时延的同时,大幅减少对内存资源的依赖。

DiskANN 的两组对比优势

① 对比基于内存的算法(如 HNSW、IVF)

- 在资源消耗和可扩展性上优势明显:十亿级向量用 HNSW 可能需要数百 GB 内存,DiskANN 用一块 SSD + 少量内存即可,以更低的资源消耗提供相近的查询性能。

② 对比基于聚类压缩的算法(如 IVF_PQ)

- IVF_PQ 靠激进压缩省内存,但压缩会拉低召回率;
- DiskANN 因为磁盘上保留了原始精度的数据用于最终校验,所以在保持高性能的同时避免了因压缩导致的召回率下降。

📌 比喻:IVF_PQ 是"把所有档案压缩成摘要卡,查谁都只看摘要"(省地方但偶尔认错人);DiskANN 是"手边放摘要卡导航,认人前一定去仓库核对原件"(既省地方又不认错人)。

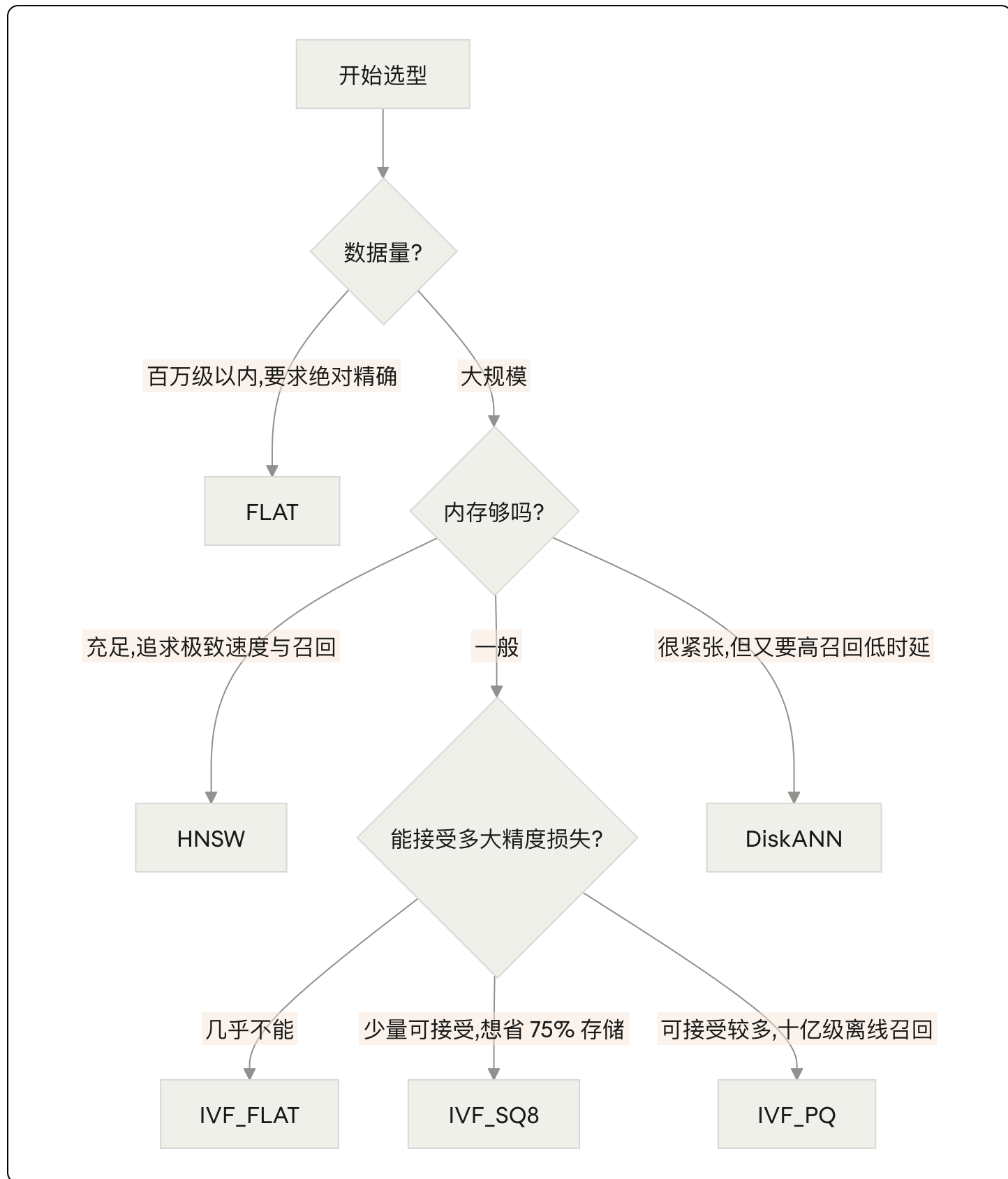
👉 适用于:十亿级超大规模数据集、内存预算有限、但仍要求高召回与低延迟的场景。

7. 总览对比与选型

7.1 六种算法横向对比

算法	核心机制	精度/召回	查询速度	内存占用	比喻
FLAT	暴力线性扫描	★★★★★ 100%	❌ 慢	高(原始向量)	挨家挨户敲门
IVF_FLAT	聚类分簇 + 簇内精确搜索	★★★★	✅ 快	高(原始向量)	分区排查
IVF_SQ8	IVF + 标量量化(4B→1B)	★★★	✅ 更快	中(↓75%)	分区 + 摘要卡
IVF_PQ	IVF + 乘积量化 (codebook)	★★	✅ 很快	低(压缩激进)	分区 + 拆段编码
HNSW	分层小世界图导航	★★★★★	⚡ 极快	❌ 很高(图 + 向量)	高速→省道→小巷
DiskANN	内存轻量索引 + 磁盘图/原始数据	★★★★	✅ 快(低时延)	✅ 很低	地图在手,档案在库

7.2 一条决策思路



7.3 记忆口诀

- **FLAT** 最老实:一个一个比。
- **IVF** 三兄弟:先分区——FLAT 不压缩、SQ8 压数字、PQ 压整段。
- **HNSW** 走图:层层导航,从粗到细。
- **DiskANN** 分家:索引住内存,数据住磁盘。

本笔记基于个人原始笔记整理,术语与参数($ANNS$ / $nprobe$ / $codebook$ / SQ / PQ 等)与原笔记保持一致,便于对照复习。